



Universidade de Brasília – UnB
Faculdade UnB Gama – FGA
Engenharia de Software

**Adequação de funcionalidades da plataforma
Brasil Participativo: Uma abordagem baseada
no uso de estratégias de desenvolvimento
orientadas à busca de desempenho**

Autor: Victor Jorge da Silva Gonçalves
Orientador: Prof. Dr. Renato Coral Sampaio

Brasília, DF
2023



Victor Jorge da Silva Gonçalves

**Adequação de funcionalidades da plataforma Brasil
Participativo: Uma abordagem baseada no uso de
estratégias de desenvolvimento orientadas à busca de
desempenho**

Monografia submetida ao curso de graduação
em Engenharia de Software da Universidade
de Brasília, como requisito parcial para ob-
tenção do Título de Bacharel em Engenharia
de Software.

Universidade de Brasília – UnB

Faculdade UnB Gama – FGA

Orientador: Prof. Dr. Renato Coral Sampaio

Brasília, DF

2023

Victor Jorge da Silva Gonçalves

Adequação de funcionalidades da plataforma Brasil Participativo: Uma abordagem baseada no uso de estratégias de desenvolvimento orientadas à busca de desempenho/ Victor Jorge da Silva Gonçalves. – Brasília, DF, 2023-

60 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. Renato Coral Sampaio

Trabalho de Conclusão de Curso – Universidade de Brasília – UnB
Faculdade UnB Gama – FGA , 2023.

1. otimização. 2. desempenho. I. Prof. Dr. Renato Coral Sampaio. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. Adequação de funcionalidades da plataforma Brasil Participativo: Uma abordagem baseada no uso de estratégias de desenvolvimento orientadas à busca de desempenho

CDU 02:141:005.6

Victor Jorge da Silva Gonçalves

Adequação de funcionalidades da plataforma Brasil Participativo: Uma abordagem baseada no uso de estratégias de desenvolvimento orientadas à busca de desempenho

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Trabalho aprovado. Brasília, DF, 19 de dezembro de 2023 – Data da aprovação do trabalho:

Prof. Dr. Renato Coral Sampaio
Orientador

Prof. Dr. Maurício Serrano
Convidado 1

Prof. Dr. John Lenon Cardoso
Gardenghi
Convidado 2

Brasília, DF
2023

*Dedico este trabalho à minha mãe, que sempre me apoiou e ajudou
durante minha trajetória, sendo a pessoa mais importante em todo o processo.*

Agradecimentos

Agradeço ao Prof. Dr. Renato Coral Sampaio por todo o apoio, gentileza e paciência durante o processo de orientação.

Agradeço à Prof. Dr. Carla Rocha pelo apoio e oportunidade de me aprofundar no tema através da minha participação no projeto do Brasil Participativo.

Resumo

Aplicações web modernas geralmente se dividem em três grandes partes: *back-end*, *front-end* e banco de dados. *Frameworks* baseados no padrão de projeto como o *Model View Controller* (MVC) podem experienciar de problemas de escalabilidade e desempenho, principalmente em relação ao uso intensivo do banco de dados. Um caso real, a plataforma Brasil Participativo, atingiu um número relativamente alto de usuários (um milhão e quatrocentos mil), o que revelou diversos pontos de fraqueza e gargalos na arquitetura desenvolvida e na abordagem adotada no desenvolvimento da *gem Decidim*, construída em cima do *framework Ruby on Rails*. Este trabalho tem por objetivo descrever um processo de refatoração da aplicação, utilizando-se de abordagens e técnicas já conhecidas de cache de dados. Para tal, será realizada uma pesquisa sobre as principais formas de utilização de tecnologias de armazenamento de informações em cache utilizando o *framework Ruby on Rails*, e estratégias de utilização que melhor beneficiem a aplicação. Em seguida serão realizados testes para verificar o impacto causado no desempenho da aplicação mediante a inserção do uso de cache.

Palavras-chave: Aplicações Web. Padrão de Projeto MVC. Ruby on Rails. Cache.

Abstract

Modern web applications typically consist of three major parts: back-end, front-end, and database. Frameworks based on design patterns such as the Model-View-Controller (MVC) may face scalability and desempenho issues, particularly regarding intensive database usage. In a real-world scenario, the Brasil Participativo platform reached a relatively high number of users (one million and four hundred thousand), revealing various weaknesses and bottlenecks in the developed architecture and the approach taken in building the Decidim gem, built on the Ruby on Rails framework. This work aims to describe a refactoring process for the application, utilizing well-known approaches and techniques for data caching. To achieve this, research will be conducted on the primary ways of using cache storage technologies with the Ruby on Rails framework and strategies that best benefit the application. After that, tests will be done in order to verify the impact caused by introducing the cache store usage.

Key-words: Web Applications. MVC Design Pattern. Ruby on Rails. Cache.

Lista de ilustrações

Figura 1 – Comunicação entre as entidades do MVC	30
Figura 2 – Exemplo de <i>template</i> que renderiza objetos com <i>caches</i> aninhados . . .	35
Figura 3 – Inclusão da opção <i>touch</i> na declaração de relacionamento entre <i>models</i>	35
Figura 4 – Arquitetura do <i>Decidim</i>	38
Figura 5 – Relatório da controladora de comentários gerado pelo Elastic APM . .	43
Figura 6 – Captura de tela dos <i>logs</i> apresentados pelo <i>Rails</i>	53
Figura 7 – Consulta SQL para classificação dos dias com maior carga de criação de textos participativos	55
Figura 8 – Consulta SQL para classificação dos textos participativos com maior quantidade de parágrafos	56

Lista de tabelas

Tabela 1	– Módulos dos espaços participativos padrões do <i>Decidim</i>	39
Tabela 2	– Módulos dos componentes padrões do <i>Decidim</i>	39
Tabela 3	– Especificação do hardware de teste	54
Tabela 4	– Top 5 dias com maior volume de textos participativos criados	55
Tabela 5	– Top 5 textos participativos de 29/11/2024 com maior volume de pará- grafos	56
Tabela 6	– Teste da página de visualização de rascunho	57
Tabela 7	– Teste extremo da página de visualização de rascunho	57
Tabela 8	– Teste da página de visualização do texto	58
Tabela 9	– Teste extremo da página de visualização do texto	58

Lista de abreviaturas e siglas

ACID	<i>Atomicity, Consistency, Isolation, and Durability</i>
API	<i>Application Programming Interfaces</i>
DRY	<i>Don't Repeat Yourself</i>
FTP	<i>File Transfer Protocol</i>
HTML	<i>HyperText Markup Language</i>
HTTP	<i>Hypertext Transfer Protocol</i>
HTTPS	<i>Hypertext Transfer Protocol Secure</i>
JSON	<i>JavaScript Object Notation</i>
MGI	Ministério da Gestão e Inovação em Serviços Público
MVC	<i>Model View Controller</i>
ORM	<i>Object Relational Mapping</i>
PPA	Plano Plurianual Participativo
RAM	<i>Random Access Memory</i>
SMTP	<i>Simple Mail Transfer Protocol</i>
SGPR	Secretaria Geral da Presidência da República
SNPS	Secretaria Nacional de Participação Social
SQL	<i>Structured Query Language</i>
XML	<i>Extensible Markup Language</i>

Sumário

1	INTRODUÇÃO	23
1.1	Justificativa	24
1.2	Objetivos	24
2	REFERENCIAL TEÓRICO	25
2.1	Conceitos Gerais	25
2.1.1	Aplicações <i>Desktop</i>	25
2.1.2	Aplicações <i>Web</i>	26
2.1.2.1	Modelo Cliente/Servidor	26
2.2	Componentes Gerais de uma Aplicação <i>Web</i>	27
2.2.1	<i>Front-end</i>	27
2.2.2	<i>Back-end</i>	27
2.2.3	Banco de Dados	28
2.2.3.1	Banco de Dados Relacionais	28
2.2.3.2	Transações ACID	28
2.2.3.3	<i>PostgreSQL</i>	29
2.2.3.4	<i>Redis</i>	29
2.3	Padrão de Projeto <i>Model-View-Controller</i>	29
2.3.1	<i>Model</i>	29
2.3.2	<i>View</i>	30
2.3.3	<i>Controller</i>	30
2.4	<i>Ruby on Rails</i>	30
2.4.1	<i>Active Record</i>	31
2.4.1.1	<i>Active Record como uma interface de ORM</i>	31
2.4.2	<i>Active Model</i>	31
2.4.3	<i>Action View</i>	31
2.4.3.1	<i>Templates</i>	32
2.4.3.2	<i>Partials</i>	32
2.4.4	<i>Action Controller</i>	32
2.5	Desempenho e Escalabilidade	32
2.6	Cache com <i>Ruby on Rails</i>	33
2.6.1	Tipos de cache no <i>Ruby on Rails</i>	33
2.6.1.1	Page Caching	33
2.6.1.2	Action Caching	34
2.6.1.3	Fragment Caching	34

2.6.1.4	Russian Doll Caching	35
2.6.2	Bases de dados para cache no Ruby on Rails	36
2.6.2.1	Memory Store	36
2.6.2.2	File Store	36
2.6.2.3	Mem cache Store	37
2.6.2.4	Redis cache Store	37
2.7	Decidim	37
2.7.1	Arquitetura do Decidim	37
2.7.1.1	Participatory Spaces	38
2.7.1.2	Components	39
2.8	Considerações Finais do Capítulo	39
3	PROPOSTA	41
3.1	Contextualizando a Plataforma Brasil Participativo	41
3.2	Principais Problemas de Desempenho da Plataforma Brasil Participativo	42
3.2.1	Sobreutilização do Banco de Dados e Indisponibilidade	42
3.2.2	Ausência do Uso de Índices em Consultas Ofensoras na Plataforma	42
3.2.3	Memory Leak do Processo Ruby	43
3.2.4	Lentidão em Páginas sob Condições Específicas	43
3.3	Proposta de Intervenção na Plataforma Brasil Participativo	44
3.3.1	Texto Participativo como Objeto de Estudo	44
3.3.2	Resultados Esperados	45
3.4	Considerações Finais do Capítulo	45
4	METODOLOGIA	47
4.1	Síntese do Referencial Teórico	47
4.2	Ferramentas	47
4.2.1	Visual Studio Code	47
4.2.2	Git	47
4.2.3	Chrome DevTools	48
4.2.4	Yarn	48
4.2.5	RuboCop	48
4.2.6	Nokogiri	48
4.2.7	Jodit	48
4.2.8	LibreOffice Writer	48
4.2.9	Webpack e Webpacker	49
4.2.10	Notion	49
4.3	Abordagem	49
4.3.1	Abordagem Organizacional	49

4.3.2	Abordagem Técnica	49
4.4	Compilação de Resultados	50
5	DESENVOLVIMENTO	51
5.1	Organização do Desenvolvimento	51
5.2	Requisitos Iniciais	51
5.3	Contextualização da Ferramenta de Textos Participativos	51
5.4	Etapas de Desenvolvimento	52
5.4.1	Sprint 1	52
5.4.1.1	Requisito 1 da Sprint	52
5.5	Testes e Resultados	53
5.5.1	Teste de Carga de Visualização do Rascunho	55
5.5.1.1	Contexto	55
5.5.1.2	Execução e Resultados	57
5.5.1.3	Testes Extremos	57
5.5.2	Teste de Carga de Visualização do Texto	57
5.5.2.1	Contexto	57
5.5.2.2	Execução e Resultados	58
5.5.2.3	Testes Extremos	58
	 REFERÊNCIAS	 59

1 Introdução

As aplicações *web* desempenham um papel crucial no cenário digital, oferecendo funcionalidades acessíveis através de navegadores *web* sem a necessidade de instalação local. Essas se diferenciam das aplicações *desktop standalone*, que requerem instalação no dispositivo do usuário para operar. Enquanto as aplicações *desktop* são autossuficientes e independem de uma conexão constante com a internet, as aplicações *web* dependem da comunicação com servidores remotos para processar dados e oferecer serviços (TANENBAUM; KLINT; BOHM, 1978).

O funcionamento de uma aplicação *web* envolve um modelo cliente-servidor, onde o cliente (navegador *web*) solicita recursos ou serviços ao servidor, que processa essas solicitações e retorna os resultados. Em termos de arquitetura, as aplicações *web* são divididas em *back-end*, *front-end* e banco de dados. O *back-end* é responsável pelo processamento das solicitações do cliente, o *front-end* lida com a apresentação e interação do usuário, enquanto o banco de dados armazena e recupera os dados necessários para a aplicação (GOUGH; BRYANT; AUBURN, 2021).

Na construção de aplicações *web*, são empregados padrões de projeto para organizar e estruturar o código de forma eficiente. Um desses padrões é o Modelo-Visão-Controlador (MVC), que divide a aplicação em três componentes principais: a *model* (responsável pelos dados e regras de negócios), a *view* (responsável pela apresentação dos dados ao usuário) e a *controller* (gerencia as interações entre o Modelo e a Visão) (MODEL-VIEW-CONTROLLER..., 2009). Um exemplo notável de aplicação *web* que segue o padrão MVC é o *Ruby on Rails*, um *framework* construído sobre a linguagem de programação *Ruby* (THE RAILS FOUNDATION, 2023h). Essa abordagem contribui para a modularidade, manutenção e escalabilidade das aplicações *web*.

Apesar dos benefícios proporcionados pelo padrão MVC na construção de aplicações *web*, é importante destacar que esses sistemas podem enfrentar desafios significativos de escalabilidade, especialmente em cenários de alto tráfego e grande quantidade de usuários. Um exemplo notável é a plataforma Brasil Participativo, do governo federal, que atualmente conta com um milhão e seiscentos mil (1.600.000) usuários. Esta plataforma, desenvolvida utilizando o *framework Ruby on Rails* e a *gem/framework Decidim*, tem enfrentado problemas de desempenho devido ao uso intensivo de suas funcionalidades e aumento substancial na demanda e acesso de usuários.

O crescimento do número de usuários pode sobrecarregar a capacidade do sistema em processar simultaneamente um grande número de solicitações, resultando em lentidão, tempo de resposta prolongado e possíveis falhas na execução de funcionalidades críticas.

Não apenas o volume de acessos pode comprometer a correta operação da plataforma, mas também o volume de dados utilizados em suas funcionalidades. Problemas como esses são indicativos da necessidade de otimização e reestruturação da arquitetura para lidar com o aumento de carga e garantir uma experiência de usuário eficiente e sem interrupções. Esses desafios ressaltam a importância de abordagens proativas na identificação e resolução de questões de desempenho, destacando a relevância de estratégias como a implementação eficaz de cache, otimização de consultas ao banco de dados e reorganização da construção do *front-end*.

1.1 Justificativa

A plataforma Brasil Participativo tornou-se a maior instância do *Decidim* em todo o mundo. Mesmo a comunidade responsável pelo desenvolvimento dessa tecnologia reconheceu que o *framework* não foi desenvolvido com pensamento de tomar proporções tão grandes quanto as que se vêem no Brasil. Dessa maneira, notou-se a necessidade de intervenção na arquitetura da plataforma para garantir que esta subsistirá mediante a carga de uso atual e futura no país.

1.2 Objetivos

Este trabalho tem como objetivo explorar oportunidades de reestruturação de funcionalidades que apresentam gargalos de desempenho e que ainda não se adequam ao contexto em que a aplicação se encontra: utilização no cenário nacional seguindo diretrizes de plataformas governamentais brasileiras. Entende-se que os resultados e a metodologia deste trabalho, apesar de serem direcionados para a plataforma Brasil Participativo, sejam replicáveis para outras plataformas *web* que utilizam do modelo arquitetural MVC, e, especialmente, *Ruby on Rails*. Os objetivos gerais, contudo, podem ser sumarizados da seguinte maneira:

- Listar e discutir alguns dos principais problemas de desempenho da plataforma;
- Escolher como foco de trabalho uma funcionalidade ou um conjunto de funcionalidades logicamente interligadas;
- Definir estratégias de otimização de desempenho da funcionalidade escolhida;
- Realizar o desenvolvimento de ajustes de acordo com as necessidades dos usuários-chaves e envolvidos da plataforma;
- Priorizar caminhos de desenvolvimento aderentes às estratégias de desempenho levantadas;

-
- Testar a solução adotada e expor os ganhos de desempenho em relação à versão anterior da aplicação;

2 Referencial Teórico

Este capítulo possui como propósito expor e definir tópicos pertinentes sobre o assunto abordado nesse trabalho, a fim de trazer maior contexto e entendimento sobre o universo do assunto. Os conceitos que aqui serão abordados são: Conceitos Gerais (Aplicações *Standalone* e Aplicações *Web*), Componentes Gerais de uma aplicação *Web* (*Front-end*, *Back-end* e Banco de Dados), padrão de projeto *Model View Controller* (MVC), *Ruby on Rails*, Escalabilidade e desempenho, e Cache com *Ruby on Rails*.

Com a clarificação desses assuntos, será traga maior fundação teórica para que se siga com a pesquisa e com o caso de estudo, possibilitando assim maior embasamento durante as discussões referentes à problemática a ser estudada.

2.1 Conceitos Gerais

A seção Conceitos Gerais tem como objetivo introduzir conceitos fundamentais no escopo deste trabalho, a saber Aplicações *Standalone* e Aplicações *Web*. A apresentação destes dois tópicos permite com que sejam destacados aspectos pertinentes ao funcionamento de um *software*, bem como a aplicação Brasil Participativo, através da comparação direta desses dois principais modelos de *software* que operam no dia a dia, desde o computador pessoal de um estudante até os grandes *mainframes* do mercado.

2.1.1 Aplicações *Desktop*

Aplicações *Desktop*, também chamadas de aplicações *standalone* são softwares que atingem seu proposito de operacionalidade sem a necessidade de estarem conectados à internet, isto é, estas aplicações rodam em um computador ou dispositivo local que não necessariamente estão conectados à *web*. Uma aplicação *desktop* é autocontida, portanto, todo o código e os recursos utilizados por ela se encontram presentes no dispositivo que a executa.

Geralmente, aplicações *desktop* são específicas à plataforma no qual foi projetada para operar, seja Windows, MacOS ou Linux. Dessa maneira, outro conceito pertinente que surge em meio às aplicações *desktop* é a portabilidade. Portabilidade se refere à possibilidade de uma aplicação ser executada em diferentes ambientes com a mínima quantidade de ajustes que for necessária. Um *software* dito portátil pode executar, por exemplo, em Windows e MacOS com pouca ou nenhuma adaptação em seu código fonte, enquanto que, um software dito pouco portátil, pode precisar ser reescrito do absoluto zero

ou ter boa parte do seu código fonte adaptado para rodar em uma plataforma diferente da qual foi primariamente projetado ([TANENBAUM; KLINT; BOHM, 1978](#)).

2.1.2 Aplicações Web

Aplicações *Web*, diferentemente de aplicações *desktop*, são executadas no dispositivo do usuário através de outra aplicação, o *Web Browser*. Aplicações *Web* funcionam através do uso do modelo cliente/servidor, onde um dispositivo remoto, que possui os recursos da aplicação, os fornece para o dispositivo cliente, que serão exibidos para o usuário pelo *browser*. Para que uma aplicação *web* execute, é necessário que o dispositivo cliente esteja conectado à internet, ou, no mínimo à uma rede que tenha acesso ao dispositivo servidor. Geralmente, a conexão estabelecida no modelo cliente/servidor é através do protocolo *Hypertext Transfer Protocol* (HTTP) ou sua variante *Hypertext Transfer Protocol Secure* (HTTPS) ([CONALLEN, 1999](#)).

2.1.2.1 Modelo Cliente/Servidor

No modelo cliente/servidor, diferentemente de aplicações *desktop* convencionais, o processamento de dados é feito no lado servidor. Portanto, na maioria das vezes, o lado cliente fica isento do processamento pesado de dados, sendo responsável apenas por exibir o resultado final ao usuário. Em geral, o fluxo seguido por uma aplicação cliente/servidor é iniciado pelo dispositivo cliente (usuário final), que solicita ao dispositivo servidor algum recurso; o servidor processa a requisição do cliente realizando o processamento necessário, e em seguida, devolve o recurso em um formato que o lado cliente consiga processar e exibir ao usuário final. Essa comunicação pode ocorrer através de diversos protocolos: *File Transfer Protocol* (FTP) para transferência de arquivos, *Simple Mail Transfer Protocol* (SMTP) para envio e recebimento de e-mails, e o já mencionado HTTP.

Cada protocolo dispõe de particularidades que derivam dos seus propósitos, fazendo com que cada um deles seja aplicável em um contexto específico. Entretanto, uma aplicação pode se beneficiar do uso de vários protocolos diferentes para cada uma de suas funcionalidades, isso a depender do propósito de cada um de seus módulos. Uma aplicação *web* moderna, geralmente dispõe de funcionalidades para gerenciar recursos e exibi-los aos usuários utilizando o protocolo HTTP, em contrapartida, a própria entidade usuário pode ser considerada um recurso, onde, cada usuário tem um e-mail vinculado, permitindo que a aplicação envie informações utilizando o protocolo SMTP em algum outro momento oportuno.

A adoção do modelo cliente/servidor pode proporcionar à aplicação *web* diversas características importantes, sendo a principal delas a centralização dos dados no lado servidor. Uma vez que os dados estejam centralizados no lado servidor, máquinas com mais recursos computacionais podem ser alocadas para que o processamento ocorra mais

rapidamente e de forma desacoplada do ambiente do lado cliente, permitindo maior portabilidade da aplicação (OLUWATOSIN, 2014).

2.2 Componentes Gerais de uma Aplicação Web

Uma aplicação *web* moderna em geral pode ser vista dividida em duas partes: *back-end* e *front-end*. Ao contrário de aplicações *standalone* convencionais que possuem a lógica de negócio altamente acoplada com a lógica de exibição, aplicações *web* tendem a ter uma divisão mais clara nesse aspecto, permitindo com que a lógica de negócio não necessariamente interfira na renderização. Vale notar ainda a grande semelhança dessa abordagem com o modelo cliente/servidor (GONG et al., 2020). Outro importante componente no contexto de aplicações *web* é o banco de dados, que cumpre o papel de persistir informações da aplicação. A seção Componentes Gerais de uma Aplicação Web possui como objetivo introduzir cada um desses conceitos e elucidar seus papéis, trazendo maior clareza para cada um de seus papéis e como estes influenciam no funcionamento da aplicação *web*, e consequentemente na sua desempenho.

2.2.1 Front-end

O *front-end* é a parte da aplicação *web* responsável pela exibição dos dados e da interface de usuário. Idealmente, o *front-end* lida apenas com a renderização de páginas, a lógica de *design* da aplicação, sistema de rotas de páginas e recursos estáticos (GONG et al., 2020).

2.2.2 Back-end

O *back-end* é a parte da aplicação *web* que lida com toda a lógica de negócio contida no escopo do *software*. Todos os dados, bem como o processamento desses dados é realizado nessa camada. Do ponto de vista do funcionamento da aplicação como um todo, o *back-end* possui como trabalho responder as requisições do usuário que vêm da camada do *front-end*. Uma vez que o usuário pode enviar e requisitar dados, o *back-end* deve ser capaz, através de um conjunto de métodos, processar a requisição e devolver uma resposta no menor tempo possível (ADAM; BESARI; BACHTIAR, 2019).

A comunicação entre *back-end* e *front-end* pode ser realizada de diversas maneiras e com vários protocolos. Uma das formas mais adotadas como mecanismo de comunicação em aplicações *web* modernas é a exposição de funções do *back-end* por meio de *Application Programming Interfaces* (APIs). As APIs podem ser vistas como interfaces de comunicação que visam abstrair a lógica de funcionamento por detrás de uma aplicação, expondo suas funcionalidades através de *endpoints* que podem ser acessados por

outras aplicações sem que a aplicação cliente precise saber como estas operam (GOUGH; BRYANT; AUBURN, 2021).

2.2.3 Banco de Dados

Banco de Dados, nesse contexto, uma forma reduzida do termo Sistema Gerenciador de Banco de Dados, é uma classe de *software* que lida com a manipulação e persistência de dados. Um banco de dados tem como objetivo fornecer aos seus usuários uma fonte de dados centralizada, com qualidade, integridade e segurança (FRY; SIBLEY, 1976). Existem diversos tipos de bancos de dados, porém, são relevantes para este trabalho: os bancos de dados relacionais e os bancos de dados de chave-valor.

2.2.3.1 Banco de Dados Relacionais

Bancos de dados relacionais são bancos cujo os dados são guardados no formato de tabelas, também denominadas relações. As tabelas são compostas por colunas e linhas, semelhante a planilhas, onde cada linha representa um registro, denominado tupla, na tabela. Bancos de dados relacionais permitem a criação de relacionamentos entre tabelas, sendo essa a motivação do termo "relacional". A maioria dos bancos de dados permitem manipular e consultar seus dados através do uso da linguagem de consulta *Structured Query Language* (SQL) (JATANA et al., 2012).

2.2.3.2 Transações ACID

No contexto de banco de dados, um importante conceito é o de transações. Uma transação é um agrupamento de operações realizadas pelo banco de dados, sendo a transação um conceito atômico, isto é, a menor instrução que o banco de dados realizará a mando do usuário. Existem características que um banco de dados pode ou não respeitar na sua implementação, estas são: atomicidade, consistência, isolamento e durabilidade (ACID). As propriedades ACID podem ser definidas da seguinte maneira:

- Atomicidade: as operações envolvidas em uma única transação são executadas como uma só, implicando em que, caso uma falhe, a transação como um todo será dada como falha.
- Consistência: os dados presentes no banco de dados devem permanecer consistentes após a execução da transação.
- Isolamento: estados intermediários da transação não são visíveis por outras transações concorrentes, implicando que, uma transação não interferirá em outra.

- Durabilidade: quando uma transação é executada e chega ao fim, seus efeitos são persistentes. Caso ocorra interrupções ou falhas no banco de dados, uma transação completa não será afetada.

(YU, 2009).

2.2.3.3 PostgreSQL

Segundo a documentação, o PostgreSQL é um gerenciador de banco de dados relacional *open-source* que implementa e incrementa o padrão da linguagem SQL. O PostgreSQL implementa todos os princípios ACID desde 2001, além de oferecer *plugins* que permitem o uso de outras funcionalidades não convencionais no contexto de banco de dados relacionais (GROUP, 2023).

2.2.3.4 Redis

Bancos de dados chave-valor são sistemas gerenciadores de banco de dados que armazenam seus dados de forma não relacional utilizando de chaves e valores. Cada registro consistirá em uma chave e em um valor, podendo este ser armazenado tanto na memória RAM, quanto no disco. Uma das principais características de bancos de dados chave-valor é a sua velocidade quando comparado com bancos relacionais. O *Redis* é um banco de dados chave valor *open-source* extremamente rápido que guarda seus dados em forma persistente no disco ou na memória RAM (CARLSON, 2013).

2.3 Padrão de Projeto *Model-View-Controller*

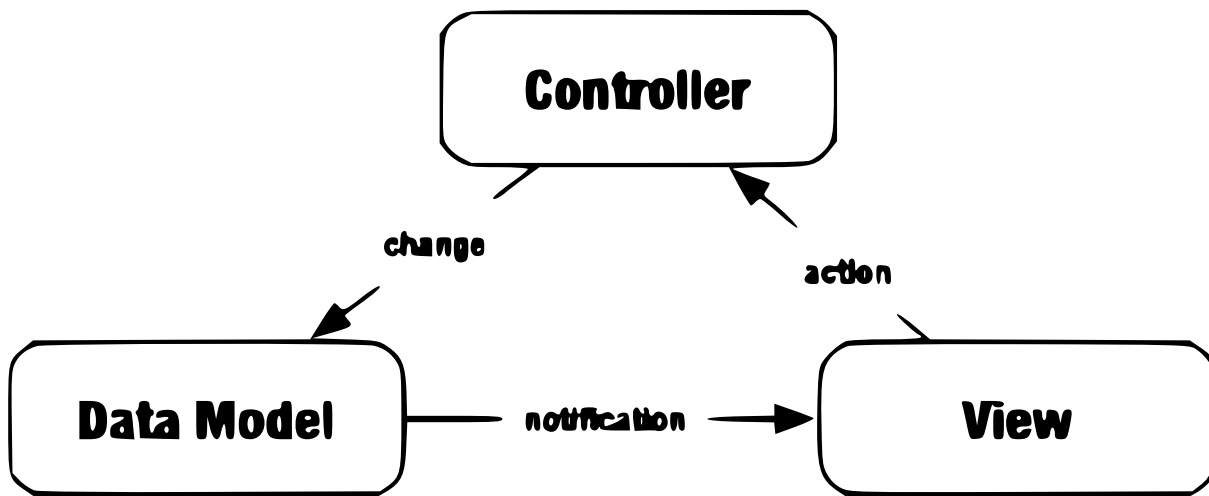
O MVC é um padrão de projeto que é considerado como um dos mais importantes na área da ciência da computação, pois este pode ser interpretado como algo mais próximo de uma filosofia de desenvolvimento de arquitetura de código, do que de um padrão de projeto que resolve instâncias de problemas de um domínio específico. O MVC possui uma grande abertura de interpretação e aplicação, podendo ser adotado desde subsistemas até aplicações inteiras. Essencialmente, o MVC é definido em três entidades: a *model*, a *view*, e a *controller* (MODEL-VIEW-CONTROLLER..., 2009).

As próximas subseções tem como objetivo descrever cada uma dessas entidades e qual papel desempenham dentro do contexto do padrão de projeto MVC.

2.3.1 Model

A *model* é a entidade do MVC que possui como preocupação encapsular informações do domínio do problema a ser resolvido pela aplicação. Um objeto de dados *model* deve saber guardar, encapsular e abstrair os dados. Vale observar também que

Figura 1 – Comunicação entre as entidades do MVC



Fonte: (MODEL-VIEW-CONTROLLER..., 2009)

cada entidade idealmente não deve extrapolar suas funções, portanto, uma *model* deve saber serializar seus dados, mas não deve saber como implementar uma funcionalidade de "salvar como" (MODEL-VIEW-CONTROLLER..., 2009).

2.3.2 View

Em poucas palavras, a *view* é a entidade do MVC que apresenta informações na tela para o usuário final. Porém, a *view* desempenha muitos outros papéis; na verdade, ela é a ponte entre o usuário e o sistema. Dessa forma, a *view* é responsável por captar o *input* do usuário e transformá-lo em comandos para o sistema processar. Outro aspecto importante é o escopo que uma *view* pode atuar. No desenvolvimento de aplicações com o modelo MVC, objetos de *view* podem ser generalistas ao ponto de saberem renderizar e lidar qualquer com qualquer tipo de texto, imagem ou mídia, ou podem ser específicos ao ponto de saberem renderizar apenas determinados objetos do sistema definidos no código pelo programador (MODEL-VIEW-CONTROLLER..., 2009).

2.3.3 Controller

A *controller* é a entidade do MVC que implementa as ações do sistema. Como já mencionado, as *models* sabem gerenciar seus dados a nível de persistência e abstração, as *views* são responsáveis por coletar o *input* do usuário para disparar as ações, estas que residem nas *controllers*. Uma ação realizada por uma *controller*, por exemplo, seria o comando "salvar como" de um dado armazenado dentro de uma *model* (MODEL-VIEW-CONTROLLER..., 2009). Em geral, a comunicação realizada pelas entidades no modelo MVC pode ser visualizada na Figura 1.

2.4 Ruby on Rails

O *Ruby on Rails*, ou simplesmente *Rails*, é um *framework* de aplicação *web* que fornece tudo o que é necessário para a criação de aplicações que seguem o modelo MVC e que utilizam de bancos de dados para armazenamento de suas estruturas e dados. O *Rails* traz para cada uma das entidades do modelo MVC uma interface que provê as funcionalidades necessárias para que estas sejam implementadas em uma aplicação *web*.

O *Rails* traz duas interfaces para que o desenvolvedor consiga criar as *models* no domínio de sua aplicação: o *Active Record* e o *Active Model*. Para as *views*, o *Rails* fornece a *Action View*, que disponibiliza funcionalidades de geração de novas *views* na aplicação. Para as *controllers*, o *Rails* fornece o *Action Controller*. Cada uma dessas interfaces são exploradas em mais detalhes a seguir ([THE RAILS FOUNDATION, 2023h](#)).

2.4.1 Active Record

O *Active Record* é o "M" do modelo MVC no contexto do *Rails*. De fato, o *Active Record* é um padrão de projeto que precede o *Rails*, este foi descrito por *Martin Fowler* em seu livro *Patterns of Enterprise Application*. No contexto de aplicações *Rails*, o *Active Record* é caracterizado por sua responsabilidade de representar regras de negócio e dados, sendo uma ponte direta entre as *models* da aplicação e o banco de dados, disponibilizando uma interface de *Object Relational Mapping* (ORM) ([THE RAILS FOUNDATION, 2023f](#)).

2.4.1.1 Active Record como uma interface de ORM

O *Active Record* provê, por padrão, uma interface de comunicação entre os objetos da aplicação com bancos de dados relacionais. Esta interface permite que o programador não necessite de escrever instruções SQL diretamente no código fonte da aplicação para buscar e persistir dados das *models* no banco de dados ([THE RAILS FOUNDATION, 2023f](#)). Além de prover esse mecanismo de interação com o banco, o *Active Record* fornece diversas outras funcionalidades que permitem a configuração de validações sobre atributos da *model*, além de prover mecanismos de *callbacks* que são ativados automaticamente para os eventos básicos do ciclo de vida do objeto: criar, salvar, atualizar e destruir.

2.4.2 Active Model

O *Active Model* possibilita incorporar funcionalidades do *Active Record* em uma classe *Ruby* pura, sem a intenção de persistir seus atributos no banco de dados. O *Rails* provê maneiras de se incluir separadamente cada uma dessas funcionalidades, permitindo com que a classe herde apenas os comportamentos desejados do *Active Record* ([THE RAILS FOUNDATION, 2023e](#)).

2.4.3 Action View

As *Action Views* no *Rails* são usadas para renderizar o resultado de uma ação ou requisição processada pelo sistema. Em termos dos fundamentos de uma aplicação cliente/servidor, a função de uma *Action View* é processar as informações a serem retornadas pelo sistema e compilá-las em um formato compreensível pelo usuário. O *Rails* facilita essa renderização por meio de três componentes chave: *templates*, *partials* e *layouts*.

2.4.3.1 Templates

Os *templates* atuam como um guia para a exibição das informações resultantes de uma ação. Por padrão, um *template* da *Action View* pode ser renderizado em diversos formatos, destacando-se entre eles: o *HyperText Markup Language* (HTML), o *JavaScript Object Notation* (JSON) e o *Extensible Markup Language* (XML).

2.4.3.2 Partials

As *partials*, ou o termo mais completo *template partials*, são exatamente o que o nome sugere: *templates* parciais, ou seja, fragmentos de código que são extraídos para um arquivo separado e que podem ser reutilizados em diversas partes. O propósito das *partials* é dividir o processo de renderização em partes para um melhor controle e permitir a reusabilidade, o que fomenta a filosofia do *Don't Repeat Yourself* (DRY) ([THE RAILS FOUNDATION, 2023b](#)).

2.4.4 Action Controller

O *Action Controller* é uma parte essencial do padrão de arquitetura MVC, representando a letra "C". Ele gerencia as solicitações, processa dados provenientes da *model* e gera a saída apropriada para exibição ao usuário. Em aplicações *Rails* convencionais, o *Action Controller* recebe a solicitação, interage com a *model* para obter ou salvar dados, e utiliza a *view* para criar uma saída em HTML ou qualquer outro formato necessário. Ou seja, o *Action Controller* atua como intermediário entre os dados das *models* e a apresentação ao usuário por meio das *Action Views*. No *Rails*, é possível definir *actions* dentro das *controllers* por meio de funções que recebem os dados da requisição e realizam as operações necessárias. Cada *action* tem um *template* de *view* correspondente, fortalecendo a cooperação entre o *Action Controller* e a *Action View* ([THE RAILS FOUNDATION, 2023a](#)).

2.5 Desempenho e Escalabilidade

A desempenho de uma aplicação *web* pode ser avaliada de várias maneiras. Duas das maneiras mais conhecidas de aferir o nível de desempenho de uma aplicação é através do número de requisições processadas por segundo, também chamado de *throughput* ou *requests/second*, e o tempo gasto pela aplicação para responder a uma requisição. Uma boa aplicação visa manter seu *throughput* alto e seu *response time* baixo, de tal forma que a experiência do usuário seja positiva, a carga de uso seja suportada e a aplicação seja confiável em momentos de pico de acesso, evitando indisponibilidade. Sabe-se que 80% do tempo gasto por uma aplicação no processamento de uma requisição é durante a execução de operações no banco de dados, e na montagem da resposta para o usuário, com a renderização de *templates* e criação do conteúdo compilado. (JUGO; KERMEK; MEŠTROVIĆ, 2014).

A aplicação de soluções web em diversas áreas de negócio, como em vendas online, destaca o conceito de escalabilidade em contextos de software. Com o aumento da demanda e do acesso de usuários, é crucial que uma aplicação web possa expandir-se mediante a adição de recursos computacionais. Embora não haja um consenso claro sobre a definição de escalabilidade, geralmente é compreendida como a capacidade de uma aplicação aumentar seu *throughput* por meio da incorporação eficiente de recursos de hardware. Essa capacidade é uma propriedade do sistema de software, fortemente influenciada pela arquitetura adotada. Se a arquitetura não permitir o aumento do *throughput* em um determinado nível de demanda com a incorporação de mais recursos, ela é considerada não escalável. (WILLIAMS; SMITH, 2004).

2.6 Cache com Ruby on Rails

Em aplicações web, especialmente no contexto do *Rails*, o conceito de *cache* refere-se à prática de armazenar conteúdos gerados durante o processamento de requisições para posterior utilização em requisições subsequentes. O *Ruby on Rails* fornece em sua API uma interface única e genérica que possibilita maneiras de integrar-se com sistemas terceiros que atuam como bases de dados para armazenamento de *cache*. Isso oferece ao programador a flexibilidade de escolher estratégias de como um dado específico será armazenado, tornando também a tarefa de definir a expiração dos dados armazenados mais simples, algo que, quando feito manualmente, é bastante suscetível a erros. Na seção 2.6.1 é explorada as maneiras que o *Rails* permite com que o programador utilize sua interface de *cache*, e na seção 2.6.2 são explicadas as principais interfaces disponibilizadas pelo *Rails* para comunicação com bases de dados para *cache* (THE RAILS FOUNDATION, 2023g).

2.6.1 Tipos de cache no Ruby on Rails

O *Ruby on Rails* fornece maneiras de se utilizar *cache* com funcionalidades embutidas, ou com *gems* que se integram com o *framework* sem muita dificuldade. A seguir são descritas cada uma dessas maneiras com suas particularidades, pontos positivos e negativos.

2.6.1.1 Page Caching

O *page caching* é uma estratégia de *cache* fornecida pela *gem* *actionpack-page_caching*, que consiste em salvar o conteúdo de uma requisição em um arquivo estático que será servido como resultado nas próximas requisições. Dessa maneira, uma vez salvo o resultado de uma requisição específica, quando realizada uma nova requisição ao mesmo *endpoint*, o arquivo salvo será servido pelo servidor, de tal forma que a requisição sequer será processada pela aplicação *Rails*. Essa abordagem apesar de simples, possui potencial para reduzir bastante o *response time* da aplicação para os *endpoints* que forem armazenados.

Porém, esta estratégia funciona apenas para requisições *get* e *head* com retorno de código 200. A utilização desse mecanismo de *cache* também não é recomendada para páginas que requeiram autenticação, pois a aplicação *Rails* precisa decidir se o remetente da requisição está ou não autorizado a receber o conteúdo solicitado (THE RAILS FOUNDATION, 2023d).

A *gem* *actionpack-page_caching* está disponível em seu repositório oficial no endereço https://github.com/rails/actionpack-page_caching.

2.6.1.2 Action Caching

O *action caching* é similar ao *page caching* no sentido de que a resposta inteira será armazenada. Essa estratégia de *cache* é fornecida pela *gem* *actionpack-action_caching*. Entretanto, nessa abordagem a requisição passa a ser recebida pela aplicação *Rails*, ao ponto que os filtros são executados antes do *cache* ser servido. Isto é, caso o *endpoint* consultado requeira autenticação para decidir se um determinado recurso será ou não servido, por exemplo, as verificações serão realizadas na *controller*, para só então o conteúdo em *cache* ser servido.

O *action caching* permite que o programador especifique diretamente o tempo de vida de um armazenamento de *cache* a nível de *actions* dentro da *controller*, além de permitir especificar condições para decidir se o resultado de uma *action* será ou não servido por *cache* (THE RAILS FOUNDATION, 2023c).

Figura 2 – Exemplo de *template* que renderiza objetos com *caches* aninhados

```
1 <!-- No template utilizado para renderização do objeto product -->
2 <% cache product do %>
3   <%= render product.games %>
4 <% end %>
5
6 <!-- No template utilizado para renderização do objeto `game` -->
7 <% cache game do %>
8   <%= render game %>
9 <% end %>
10
```

Fonte: (THE RAILS FOUNDATION, 2023g)

2.6.1.3 Fragment Caching

Aumentando ainda mais o nível de granularidade do armazenamento de informação em *cache*, o *Rails* fornece um mecanismo chamado *fragment caching*. Com o *fragment caching* é possível armazenar fragmentos específicos do *template* de uma *view*, ao invés de armazená-la por inteira. Essa abordagem é especialmente utilizada em trechos da *view* onde objetos de *models* são renderizados.

Quando o *Rails* se depara com um fragmento de *view* que exibirá informações de um objeto de uma *model*, e que há a instrução de armazenar o resultado em *cache*, uma chave única será criada com base na junção do nome da *view*, o nome da *model* do objeto em questão, o *id* e o atributo *updated_at* do objeto, que representam a chave primária e o momento em que o objeto foi atualizado pela última vez, respectivamente. Caso não exista um valor já armazenado em *cache* para esta chave, o fragmento será renderizado pela aplicação e o resultado será armazenado na memória com associação direta à essa chave (THE RAILS FOUNDATION, 2023g).

2.6.1.4 Russian Doll Caching

Ao utilizar a estratégia de *fragment caching*, nos cenários onde existem objetos cuja renderização acontece de forma aninhada com outros objetos em uma relação de *has_many/belongs_to*, se um dos objetos aninhados for alterado, ainda assim o fragmento armazenado em *cache* não expirará, pois o objeto mais externo não terá seu atributo *update_at* atualizado, conforme a Figura 2. Para resolver esse problema, o *Rails* oferece uma estratégia de armazenamento de *cache* chamada *Russian Doll Caching*. Essa estratégia consiste em adicionar no código da *model* aninhada a opção *touch: true* conforme ilustrado na Figura 3. Dessa maneira, sempre que um objeto aninhado for atualizado, o objeto mais externo também será marcado como atualizado (THE RAILS FOUNDATION, 2023g).

Figura 3 – Inclusão da opção *touch* na declaração de relacionamento entre *models*

```
1 class Product < ApplicationRecord
2   has_many :games
3 end
4
5 class Game < ApplicationRecord
6   belongs_to :product, touch: true
7 end
8
```

Fonte: (THE RAILS FOUNDATION, 2023g)

2.6.2 Bases de dados para cache no Ruby on Rails

Como mencionado anteriormente, o *Ruby on Rails* provê uma interface para comunicação com diversas tecnologias de armazenamento de *cache*. Ele permite com que o programador especifique qual tecnologia deve ser utilizada na aplicação, e fornece uma interface que provê a fundação para interação com o sistema de *cache*. Essa interface é o *ActiveSupport::cache::Store*, que fornece métodos básicos para o programador: *read*, *write*, *delete*, *exist?*, e *fetch*. Existem implementações dessa interface providas pelo *Rails*, chamadas de *Cache Store*, cujo propósito é operar com tecnologias de armazenamento conhecidas. Essas implementações são: o *Memory Store*, o *File Store*, o *Mem cache Store*, o *Redis cache Store*, e o *Null Store*; além de permitir o uso de *Cache Stores* customizados.

2.6.2.1 Memory Store

O *Memory Store* é uma tecnologia de armazenamento que utiliza a memória do próprio processo *Ruby* em execução para guardar os registros. Esse *cache store* possui as seguintes propriedades:

- O tamanho do armazenamento é fixo, e deve ser especificado na configuração da aplicação;
- Sempre que faltar espaço para inserir novos registros, uma limpeza será realizada removendo os registros que não foram utilizados recentemente;
- No caso da aplicação estar rodando em múltiplos processos, um processo não terá acesso ao *cache* do outro.

2.6.2.2 File Store

O *File Store*, como o próprio nome sugere, utiliza o próprio sistema de arquivos para guardar os registros de *cache*. Esse *cache store* possui as seguintes propriedades:

- O caminho do diretório utilizado para *cache* é especificado na configuração da aplicação;
- No caso da aplicação estar rodando em múltiplos processos no mesmo *host*, todos eles terão acesso ao mesmo *cache*.
- O volume de dados utilizado pelo armazenamento de *cache* cresce até que o disco se encontre cheio, sendo necessária a limpeza de registros antigos.

2.6.2.3 Mem cache Store

O *Mem cache Store* é uma tecnologia de armazenamento de *cache* baseada no *memcached server* da Danga. O *Rails* utiliza a *gem dalli* para operar com esse sistema de *cache*, por padrão. Esse *cache store* possui a seguinte propriedade:

- É possível utilizar um *cluster* de servidores *memcached*, porém, esses devem ser especificados na configuração da aplicação ou via variável de ambiente.

2.6.2.4 Redis cache Store

O *Redis cache Store*, como o próprio nome sugere, é um *cache store* que utiliza o *Redis* como tecnologia de armazenamento de *cache*. Esse *cache store* possui as seguintes propriedades:

- Dada a natureza do uso, é recomendado que se utilize um servidor *Redis* dedicado para *cache*.
- Servidores de *cache Redis* permitem utilizar políticas de expiração de registros por: registro utilizado com menor frequência, ou registro que não é utilizado a mais tempo.
- O *Redis cache Store* permite especificar parâmetros como: *timeout* para conexão, *timeout* para leitura, *timeout* para escrita e tentativas de reconexão.
- O tempo gasto para criar ou reescrever um registro em *cache* é pequeno, sendo as vezes mais vantajoso reescrever um registro do que esperar muito tempo para buscá-lo.

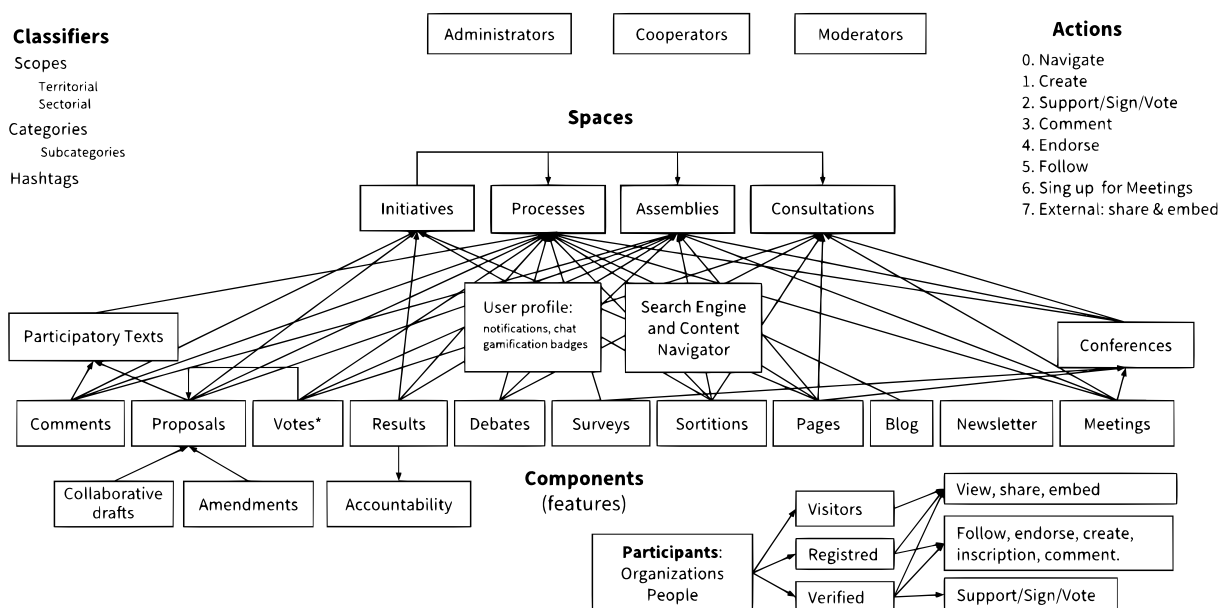
2.7 Decidim

A plataforma Brasil Participativo foi desenvolvida com o *framework Ruby on Rails* utilizando a *gem Decidim*. Segundo a própria documentação disponibilizada pela comunidade de contribuidores do *Decidim*, a *gem* é descrita como um *framework* que visa possibilitar que qualquer pessoa possa criar uma plataforma *web* para ser utilizada como rede política para participação democrática. O *Decidim* fornece a organizações a capacidade de criar processos para planejamento estratégico, orçamento participativo, concepção colaborativa de regulamentos, espaços urbanos e processos eleitorais (METADECIDIM, 2023a).

2.7.1 Arquitetura do Decidim

O *Decidim* possui várias entidades em sua arquitetura. As mais importantes para entendimento de seu funcionamento são os espaços participativos e os componentes, referidos na documentação como *participatory spaces* e *components*, respectivamente. Uma visão macro das entidades do *Decidim* pode ser visto na Figura 4. O *Decidim* armazena dados de cada uma dessas estruturas utilizando a abordagem padrão de aplicações *Rails like*, isto é, utilizando o *Active Record* e persistindo os dados em um banco de dados relacional, o *PostgreSQL*.

Figura 4 – Arquitetura do *Decidim*



Fonte: (METADECIDIM, 2023b)

Tabela 1 – Módulos dos espaços participativos padrões do *Decidim*

Espaço Participativo	Módulo
Assembleias	decidim-assemblies
Consultas ou Eleição	decidim-elections
Iniciativas	decidim-initiatives
Processos Participativos	decidim-participatory_processes

Fonte: ([METADECIDIM, 2023b](#))

2.7.1.1 Participatory Spaces

Os *participatory spaces* são *frameworks* que definem como a participação será realizada, os canais ou meios pelos quais cidadãos ou membros de uma organização podem processar solicitações ou coordenar propostas e tomar decisões. Iniciativas, processos, assembleias e consultas são todos espaços participativos, estes referidos na documentação como *initiatives*, *participatory processes*, *assemblies* e *consultations*, respectivamente. Exemplos específicos incluem: uma iniciativa cidadã para alterar diretamente uma regulamentação; uma assembleia geral ou conselho de trabalhadores; um orçamento participativo, planejamento estratégico ou processo eleitoral; um referendo ou uma convocação para votar "Sim" ou "Não" para mudar o nome de uma organização ([METADECIDIM, 2023b](#)).

A nível de código, cada implementação de um novo *participatory space* é feita em um submódulo separado em uma *gem* própria. Uma relação entre cada um dos espaços participativos já existentes por padrão no *Decidim* e o seu respectivo submódulo pode ser vista no Quadro 1.

2.7.1.2 Components

Os *components* são os mecanismos participativos que permitem uma série de operações e interações entre os usuários da plataforma dentro de cada um dos espaços participativos. No *Decidim* há disponível por exemplo os seguintes componentes: comentários, propostas, debates, pesquisas, sorteios, blogs e reuniões. Outros componentes que se baseiam nos componentes básicos são: textos participativos, responsabilidade e conferências ([METADECIDIM, 2023b](#)).

Da mesma forma que ocorre com os espaços participativos, os componentes são implementados através de submódulos em formato de novas *gems*. No Quadro 2 é possível conferir o nome do módulo de cada componente fornecido por padrão pelo *Decidim*.

TODO: explicar os participatory texts

Tabela 2 – Módulos dos componentes padrões do *Decidim*

Componente	Módulo
Comentários	decidim-comments
Propostas	decidim-proposals
Debates	decidim-debates
Pesquisas	decidim-surveys
Sorteios	decidim-sortitions
<i>Blogs</i>	decidim-blogs
Reuniões	decidim-meetings

Fonte: (METADECIDIM, 2023b)

2.8 Considerações Finais do Capítulo

Este capítulo visou esclarecer conceitos relevantes sobre aplicações *web* e seu funcionamento, a fim de permitir ser introduzida de maneira clara a abordagem de otimização de desempenho da plataforma Brasil Participativo, esta desenvolvida utilizando a *gem Decidim* que utiliza o *framework Ruby on Rails*.

Para compreender os desafios enfrentados pela plataforma e perceber a aplicabilidade das soluções propostas, é fundamental entender o funcionamento de aplicações cliente/servidor e o fluxo de operação de uma aplicação MVC, especialmente no contexto do *framework Ruby on Rails*. Como observado, grande parte do tempo consumido por uma aplicação web está relacionada à execução de operações no banco de dados e à compilação da camada de visualização para fornecer uma resposta ao cliente, etapas essas que podem ser otimizadas com a utilização de recursos de *cache* fornecidos pelo próprio *framework* ou com escolhas de arquitetura que priorizem a simplicidade do funcionamento da ferramenta.

3 Proposta

Este capítulo tem como objetivo apresentar a proposta de intervenção na plataforma Brasil Participativo, trazendo a debate alguns dos problemas de desempenho conhecidos, e esclarecendo os detalhes do que será feito. A Seção 3.1 explica o que é a plataforma, sobre como ela nasceu, seus objetivos futuros e os já alcançados. Na Seção 3.2, são expostos os desafios enfrentados pela plataforma no ano de 2023, explorando as possibilidades de intervenção para otimização. A Seção 3.3 traz as justificativas e define o foco do trabalho, elucidando a forma de intervenção a fim de melhorar o desempenho da plataforma e consequentemente satisfazer a necessidade do usuário final.

3.1 Contextualizando a Plataforma Brasil Participativo

A Plataforma Brasil Participativo é uma iniciativa do governo federal voltada para a promoção da participação social. Desenvolvida em *software* livre com o apoio de diversos parceiros, incluindo a Dataprev, a comunidade Decidim-Brasil, o Ministério da Gestão e Inovação em Serviços Públicos (MGI) e a Universidade de Brasília (UnB), ela foi criada sob a responsabilidade da Secretaria Nacional de Participação Social da Secretaria Geral da Presidência da República (SNPS/SGPR).

A plataforma tem como propósito possibilitar que a população contribua ativamente na criação e melhoria das políticas públicas. Uma de suas primeiras iniciativas foi o Plano Plurianual Participativo, assinado pela SGPR e pelo Ministério do Planejamento e Orçamento (MPO). Durante o período de 11 de maio a 16 de julho de 2023, a plataforma permitiu a coleta de propostas da sociedade e a priorização de programas e propostas para o Plano Plurianual (PPA) 2024-2027.

A participação ativa na etapa digital do PPA atingiu mais de um milhão e quatrocentos mil (1.400.000) usuários, conquistando o título de maior experiência de participação social na internet realizada pelo governo federal. A plataforma continuará evoluindo, permitindo que conselhos nacionais criem suas páginas, ministérios realizem consultas públicas e órgãos federais promovam a participação da população na definição de decretos, portarias e outras ações.

Essa abertura à participação digital representa um marco importante para a democracia, possibilitando que cidadãos influenciem diretamente nas decisões governamentais. A Plataforma Brasil Participativo oferece a oportunidade de criação de perfis individuais para facilitar a participação ativa dos interessados ([PARTICIPATIVO, 2023](#)).

A plataforma no presente momento se apoia totalmente nas funcionalidades dis-

ponibilizadas pela *gem Decidim*, preocupando-se apenas em implementar em sua instância correções de *bugs*, customizações de estilo, comportamento de regras de negócio, e pequenas funcionalidades extras. De fato, a maior parte do código executado na plataforma não é visto em seu repositório oficial, pois este está abstraído dentro da *gem Decidim*. O repositório oficial do Brasil Participativo está disponível no *Gitlab* no endereço: <https://gitlab.com/lappis-unb/decidimbr/decidim-govbr>. Já o repositório oficial do *Decidim* está disponível no *Github* no endereço: <https://github.com/decidim/decidim>.

3.2 Principais Problemas de Desempenho da Plataforma Brasil Participativo

No ano de 2023 o Brasil Participativo acumulou um milhão e quatrocentos mil usuários únicos. Tal volume de usuários implicou que a plataforma exibisse seus primeiros problemas de desempenho que foi observado pelos usuários através da indisponibilidade do serviço durante o período de alta demanda. Conforme apontado pela Dataprev, empresa responsável pela infraestrutura da aplicação no ambiente produtivo, muitos problemas de diferentes naturezas foram notados: sobreutilização do banco de dados e a indisponibilidade do mesmo, falta do uso de índices e *memory leak*.

3.2.1 Sobreutilização do Banco de Dados e Indisponibilidade

Foi notado pela Dataprev que o acesso a páginas de propostas e criação de novos usuários gera enorme utilização do banco de dados. Por exemplo, um dos problemas notados foi a utilização do comando SQL *ILIKE* para desambiguação de *nicknames* na plataforma. Comando este que requer bastante processamento do banco de dados, atingindo tempos na casa de dois segundos. Requisições que demandam tanto tempo do banco de dados, aliadas à alta demanda de usuários, fizeram com que o PostgreSQL atingisse o limite de conexões em seu *pool* em vários momentos. No ambiente produtivo, o tempo de espera limite para obtenção de uma conexão do *pool* do banco de dados é de cinco segundos, porém diversas requisições retornaram com erro para os usuários por exceder esse tempo limite. Desta forma, conclui-se que muitas ações do sistema utilizam de forma excessiva o banco de dados, sendo necessária uma estratégia de refatoração para que este problema seja resolvido.

3.2.2 Ausência do Uso de Índices em Consultas Ofensoras na Plataforma

Foi constatado pela Dataprev que as sete principais consultas mais ofensoras da plataforma poderiam ter seu desempenho melhorado através do uso de índices em suas tabelas no banco de dados. Através da observação do uso da plataforma no ano de 2023, um

padrão de consultas recorrentes foi observado. A Dataprev iniciou um trabalho interno para implementação dos índices no banco de dados. A criação desses índices pode ser proposta para a comunidade *Decidim* posteriormente, com o intuito de torná-la padrão no projeto.

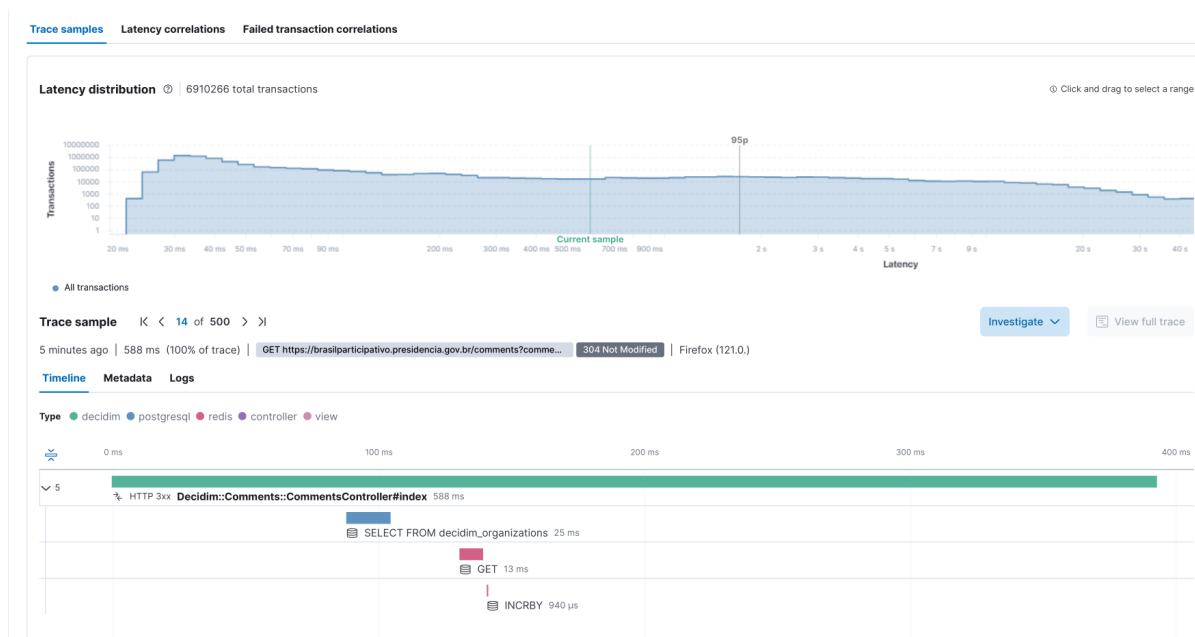
3.2.3 Memory Leak do Processo Ruby

A Dataprev observou que, com o tempo, a memória alocada para a execução da aplicação *Rails* aumenta sem dar qualquer indícios de recuo, o que implicou em um trabalho extra por parte da equipe para reiniciar a aplicação sempre que esta atingisse um determinado teto de uso de memória.

3.2.4 Lentidão em Páginas sob Condições Específicas

Durante a navegação pelas páginas da plataforma, foi notada lentidão em algumas delas. Essa lentidão ocorria na escala de segundos, e em casos mais graves na escala de minutos. Notavelmente, algumas das páginas que demonstraram baixo desempenho foram: a exibição de propostas com um alto número de comentários, e a exibição de textos participativos que possuíam algumas centenas de parágrafos em sua composição. Uma das evidências fornecidas pela Dataprev pode ser vista na Figura 5; esta demonstra o desempenho da controladora de comentários da aplicação. A ferramenta utilizada para síntese dessa informação foi o Elastic APM. Contudo, o problema não teve sua causa raiz diagnosticada pela Dataprev nessa rodada de investigação.

Figura 5 – Relatório da controladora de comentários gerado pelo Elastic APM



Fonte: Dataprev

3.3 Proposta de Intervenção na Plataforma Brasil Participativo

O Brasil Participativo diariamente recebe atualizações, estas sendo solicitadas diretamente pelos *stakeholders* da Presidência do Brasil, e coordenadas pelo time de desenvolvimento do Laboratório de Competência em Software Livre da UnB (Lab Livre). Mediante às oportunidades de intervenção no código fonte da plataforma por solicitação dos *stakeholders*, este trabalho tem como proposta, além de levantar alguns dos principais problemas de desempenho da plataforma Brasil Participativo, escolher uma funcionalidade ou conjunto de funcionalidades que já estejam cotadas para desenvolvimento como objeto de estudo principal. Para essa escolha, será mandatório que a funcionalidade tenha problemas de desempenho relatados ou pelos usuários ou pelos relatórios da Dataprev.

O objetivo é realizar os desenvolvimentos solicitados, visando satisfazer a necessidade dos interessados com uma abordagem primariamente voltada para o ganho de desempenho computacional.

3.3.1 Texto Participativo como Objeto de Estudo

Durante o ano de 2024 surgiu para o Brasil Participativo a necessidade de adequar a funcionalidade de Textos Participativos vinda do *Decidim*. Esta adequação deveria cobrir os requisitos que foram levantados pelos *stakeholders* e que posteriormente foram trabalhados pelo time de produto do Lab Livre. Destaca-se o fato de que algumas rodadas de desenvolvimento já foram feitas pelo time de desenvolvimento do Lab Livre no começo de 2024. Entretanto, um dos problemas que já era percebido mesmo na versão original da funcionalidade, mostrou-se persistente mesmo após essas entregas do time do Lab Livre, dado que o objetivo não era a completa refatoração da ferramenta.

O problema pode ser caracterizado pelo baixo desempenho computacional, onde a renderização da página de visualização e edição do texto leva minutos para ser feita, a depender da quantidade de parágrafos presentes. Outro notável problema de desempenho é a lentidão da página para responder ao comando do usuário, uma vez renderizada. Comandos como o *scroll* e clique do mouse podem levar segundos para serem processados pelo navegador devido ao alto volume de conteúdo presente na página.

Sendo assim, diante desta oportunidade, a funcionalidade escolhida como foco para este trabalho é a ferramenta de Textos Participativos. O estudo e a realização de desenvolvimentos será feita de tal forma que a ferramenta esteja aderente aos requisitos levantados, e cumpra, principalmente, o requisito de desempenho.

3.3.2 Resultados Esperados

Com a correta execução da análise, estudo e intervenção na ferramenta de Textos Participativos do Brasil Participativo, espera-se que alguns dos resultados deste trabalho sejam:

- Adição de novas funcionalidades na ferramenta de Texto Participativo;
- Satisfação dos requisitos funcionais levantados;
- Melhora perceptível no desempenho da ferramenta, reduzindo o tempo gasto pelo servidor para processar requisições do usuário;
- Aplicação das melhorias em ambiente produtivo do Brasil Participativo.

Apesar do foco do estudo ser o cumprimento dos objetivos listados acima, entende-se também que um dos frutos deste trabalho é a exposição da metodologia de análise e desenvolvimento de funcionalidades em aplicações *web* MVC com preocupação em desempenho, especialmente *Ruby on Rails*. É esperado que as etapas de trabalho aqui exploradas sejam reproduzíveis para outros casos semelhantes onde se há a necessidade de realizar o desenvolvimento cumprindo com requisitos de desempenho mais rigorosos.

3.4 Considerações Finais do Capítulo

Neste capítulo, foram expostos alguns dos problemas encontrados na plataforma Brasil Participativo. Destacou-se a característica de seu uso no contexto do Brasil, e seu impacto no cenário de participação social e utilização da tecnologia. Considerando o cenário de desenvolvimento contínuo da plataforma sob direção do Lab Livre da UnB, a proposta central deste trabalho foi firmada sobre o desenvolvimento de melhorias e adaptação da funcionalidade de Textos Participativos. Esta funcionalidade hoje apresenta gargalos de desempenho, além da necessidade de adequação para as necessidades dos usuários. A execução dessas atividades visam não apenas a melhoria do desempenho da aplicação, mas também a garantia de uma experiência mais satisfatória para os usuários da plataforma Brasil Participativo, possibilitando maior facilidade na participação digital.

4 Metodologia

Este capítulo explica a metodologia adotada neste trabalho, descrevendo suas etapas, ferramentas utilizadas e os perfis envolvidos na resolução da problemática central. O trabalho segue o formato de pesquisa aplicada mista.

4.1 Síntese do Referencial Teórico

A fundamentação teórica presente neste trabalho foi feita com objetivo de trazer o conhecimento necessário para completo entendimento dos tópicos envolvidos na solução do problema de desempenho e adequação da ferramenta de Textos Participativos do Brasil Participativo. Os tópicos envolvidos na execução deste trabalho incluem desde técnicas organizacionais de desenvolvimento de *software* até a abordagens de desenvolvimento sob o rigor de requisitos de desempenho mais restritos, especialmente com o uso do *framework Ruby on Rails*.

4.2 Ferramentas

Para a execução desse trabalho foi adotado o uso de diversas ferramentas:

4.2.1 Visual Studio Code

O Visual Studio Code (VSCode) é um *software* editor de textos de código aberto amplamente utilizado para codificação de sistemas e aplicações. No VSCode é possível trabalhar com o repositório de código da aplicação de forma fácil e integrada, com capacidades de *syntax highlighting*, *auto-formatting* e visualizações do gerenciador de versões Git. Esse editor ainda conta com uma grande gama de extensões da comunidade para facilitar e melhorar a qualidade do desenvolvimento. Uma das extensões utilizadas foi o RubyLSP, que permite a verificação do código em tempo real, capturando erros de sintaxe, além de fornecer um mecanismo de crítica do código considerando as regras de *lint* definidas no projeto.

4.2.2 Git

O Git é uma ferramenta gratuita de código aberto de gestão de versionamento distribuído de código, que pode ser utilizado desde pequenas até grandes aplicações. O Git é amplamente utilizado para gerenciamento de projetos modernos de código.

4.2.3 Chrome DevTools

O Chrome DevTools é uma ferramenta fornecida no navegador Google Chrome para visualização e manipulação de código das páginas abertas. Essa ferramenta permite com que o desenvolvedor descubra problemas de HTML, JavaScript de forma interativa utilizando o terminal que permite a execução de código, além de permitir a inspeção de elementos CSS, visualização de dados como imagens salvas, cookies, resultados de requisições e desempenho.

4.2.4 Yarn

O Yarn é um gerenciador de pacotes de código aberto para gerenciar a distribuição de dependência em projetos JavaScript. Seu objetivo é ajudar no processo de instalação, atualização, configuração e remoção de dependências de pacotes.

4.2.5 RuboCop

O RuboCop é um analisador de estilo de código e formatador *Ruby* que se baseia no guia de estilo de código *Ruby Style Guide*, que é guiado pela comunidade de desenvolvedores. O uso do RuboCop juntamente do RubyLSP acelerou o desenvolvimento e a legibilidade do código, trazendo em tempo real sugestões de refatoração de acordo com os padrões da comunidade.

4.2.6 Nokogiri

O Nokogiri é uma *gem Ruby* usada para manipular documentos HTML e XML. A biblioteca conta com uma conjunto grande de funções que auxiliam o processamento de HTML, permitindo com que o desenvolvedor não precise se preocupar com problemas específicos de representação de XML e HTML no *Ruby*.

4.2.7 Jodit

O Jodit é um editor de textos para navegadores. Este editor segue o formato *What You See is What You Get* (WYSIWYG), que basicamente permite com que o usuário insira o texto com a formatação desejada e com a confiança de que essa formatação será persistida e exibida posteriormente.

4.2.8 LibreOffice Writer

O LibreOffice Writer é uma ferramenta de edição de textos de código aberto similar o Microsoft Word. O LibreOffice Writer permite a escrita de textos, exportação em formato PDF e afins. O LibreOffice Writer é utilizado para trabalhar com os arquivos do

formato *docx*, que é utilizado para a primeira confecção dos Textos Participativos pelos usuários.

4.2.9 Webpack e Webpacker

O Webpack é uma ferramenta de empacotamento escrita em JavaScript usada para realizar a transformação de ativos servidos pela aplicação para um formato compreendido pelos navegadores web. Os ativos da aplicação podem incluir desde imagens, arquivos de estilo até código JavaScript. O Webpacker, por sua vez, é uma extensão disponível para o *Ruby on Rails* que realiza a interface de comunicação entre a aplicação, que é escrita em Ruby, com o Webpack.

4.2.10 Notion

Em geral, o Notion pode ser definido como uma aplicação *web* utilizada para anotações. O Notion conta com a possibilidade de se trabalhar em conjunto em espaços de trabalho colaborativos. O Notion pode servir como um repositório de ideias e anotações na nuvem, facilitando a recuperação e organização.

4.3 Abordagem

A abordagem deste trabalho pode ser dividida em duas vertentes: a abordagem organizacional, e a abordagem técnica. A abordagem organizacional diz respeito à forma como o trabalho foi conduzido durante o tempo, e a divisão das atividades. A abordagem técnica demonstra como os problemas de *software* foram tratados para se chegar no resultado esperado.

4.3.1 Abordagem Organizacional

A fim de alcançar os objetivos destacados será adotada a metodologia de desenvolvimento de *software* ágil (*agile*). A rotina de execução de trabalho será dividida em *sprints* de uma semana cada, onde em cada uma delas um conjunto de funcionalidades e atividades serão realizadas. No fim de cada *sprint* o que foi feito é validado com o time de produto e se necessário com os *stakeholders* do projeto.

4.3.2 Abordagem Técnica

A primeira atividade é a análise de desempenho da aplicação. Nessa é feita a instalação da aplicação no computador de trabalho para que se tenha a liberdade de utilizar a aplicação da forma que se queira. A aplicação é executada localmente para realizar os

testes com a funcionalidade. Os testes são realizados tentando simular as mesmas condições do ambiente produtivo, onde o erro foi detectado pelos usuários. Quando o resultado for alcançado, isto é, o comportamento de baixo desempenho, o próximo passo é a análise do processamento da requisição através da própria ferramenta de *logs* do *framework*. Uma boa ferramenta de logs fornecerá ao desenvolvedor detalhes de tempo gasto por cada componente chave do sistema.

Uma vez identificadas as possíveis causas do gargalo no sistema, como por exemplo: métodos lentos, acesso excessivo ao banco de dados e mal uso de memória, o desenvolvedor realiza ajustes pontuais visando reduzir o impacto causado naquele ponto de execução do código.

Depois de realizado o ajuste, volta-se para a primeira etapa, que é analisar novamente o desempenho da aplicação. No fim do dia, a metodologia pode ser explicada como um ciclo de análise e ajustes. É imprescindível também que o desenvolvedor saiba qual tipo de ajuste aplicar para um determinado problema. Ao decorrer do trabalho também será demonstrado como isso pode ser feito. O ciclo de ajuste e análise termina quando os requisitos funcionais de tempo forem satisfeitos.

4.4 Compilação de Resultados

Como artefato final deste trabalho, será feita a compilação dos resultados obtidos com os ajustes realizados na plataforma do Brasil Participativo sobre o uso da ferramenta de Textos Participativos. Para demonstrar de forma quantitativa os resultados alcançados, será feita uma tabela comparando os dados de desempenho da aplicação em testes de simulação realizados com a ferramenta JMeter.

5 Desenvolvimento

Este capítulo tem como objetivo apresentar de forma detalhada a forma como se deu o desenvolvimento dos ajustes na funcionalidade de Texto Participativo. O desenvolvimento foi caracterizado pela sua forma incremental inspirado na metodologia ágil; na Seção 5.1 é explicada as principais características da metodologia de trabalho. Uma série de requisitos que são discutidos na Seção 5.2 foram levantados para compor o ponto de partida do desenvolvimento. Uma breve contextualização do estado inicial da funcionalidade na plataforma é feito na Seção 5.3. A Seção 5.4 traz uma visão cronológica das atividades realizadas, descrevendo a abordagem utilizada e a tomada de decisões realizada em cada momento. Na Seção 5.5 são apresentados os resultados obtidos, tanto de forma quantitativa quanto qualitativa.

5.1 Organização do Desenvolvimento

A organização do trabalho seguiu um modelo inspirado no desenvolvimento ágil, onde novas alterações eram feitas a cada *sprint* e validada com o time de produto do Lab Livre e com os *stakeholders* do projeto. Ao final de cada *sprint*, após a validação do desenvolvimento realizado, com base nos *feedbacks* do time de produto o desenvolvimento seguia para um novo objetivo: corrigir e alterar algo ou adicionar mais funcionalidades.

5.2 Requisitos Iniciais

O desenvolvimento teve início com um conjunto de requisitos levantados pelos *stakeholders* da Presidência que foram posteriormente trabalhados pelo time de produto do Lab Livre. No contexto político que se encontrava a plataforma Brasil Participativo, havia, principalmente, a necessidade de trazer aspectos de uma funcionalidade semelhante da plataforma Participa+ Brasil, que também é uma plataforma de participação social do governo federal. No Participa+ Brasil, o usuário administrador submetia o texto copiando e colando.

*

5.3 Contextualização da Ferramenta de Textos Participativos

O texto participativo.

5.4 Etapas de Desenvolvimento

No desenvolvimento dos ajustes e melhorias na ferramenta de textos participativos, no total houveram quatro *sprints* de desenvolvimento, com duração média de um mês. Cada uma delas foi guiada pelos requisitos específicos levantados para aquele momento de trabalho, além dos *feedbacks* fornecidos pelos usuários e pelo time de produto do Lab Livre.

5.4.1 Sprint 1

A *Sprint* 1 pode ser caracterizada pelo início dos trabalhos, onde houve foco nos três requisitos que eram centrais na proposta de melhoria do componente:

- **Requisito 1 da Sprint:** Diminuir o tempo de renderização da página de visualização do texto participativo;
- **Requisito 2 da Sprint:** Possibilitar a criação de textos participativos através de um editor de textos próprio do Brasil Participativo, contando com o uso do comando copiar e colar, trazendo texto de fontes como o Microsoft Word e Google Docs;
- **Requisito 3 da Sprint:** Refatoração da página de visualização do rascunho (edição de parágrafos), trazendo-a para o mesmo padrão de *design* que a tela de visualização.

De acordo com o time de produto do Lab Livre, o objetivo central era fazer um *Minimum Viable Product* (MVP) para servir como uma espécie de prova de conceito, onde analisariam os resultados obtidos para eventualmente tomar uma decisão de manter ou não a mesma estratégia, a fim satisfazer as necessidades levantadas pelos *stakeholders*. As próximas seções revelam detalhes da abordagem utilizada no desenvolvimento de cada requisito que foi listado acima.

5.4.1.1 Requisito 1 da Sprint

Aplicações *Ruby on Rails* possuem um sistema de *logs* que permitem que o desenvolvedor obtenha informações da execução de vários tipos e em vários níveis de detalhamento, sendo eles:

- *debug*;
- *info*;
- *warn*;
- *error*;

- *fatal*;
- *unknown*.

Por padrão, aplicações *Rails* executadas em modo de desenvolvimento terão seu nível de *log* como *debug*, e quando executada em modo produtivo será *info* (THE RAILS FOUNDATION, 2025).

Na condução dessa análise, o Brasil Participativo foi executado em modo desenvolvimento com nível de *logs* como *debug*. Com esse nível de informações, é possível visualizar na saída padrão do Puma quais consultas SQL estão sendo executadas, quais *views* estão sendo renderizadas pelo *Rails* durante a requisição e acessos ao *ActiveStorage*. Alguns desses registros são acompanhados pelo tempo gasto na execução e em alguns casos a quantidade de alocações também é informada.

Para realizar o primeiro diagnóstico, foi criado um texto participativo com em torno de 600 parágrafos, em seguida foi acessada a página de visualização. A Figura 6 mostra os *logs* apresentados pela aplicação no processamento da requisição. Nota-se que a maior parte do tempo foi gasta com a renderização da *views* e suas *partials*, correspondendo a 96,7% do tempo utilizado para processar toda a requisição.

Figura 6 – Captura de tela dos *logs* apresentados pelo *Rails*

```
[2025-07-17 13:14:21 -0300] DEBUG --    Rendered /home/victor/.asdf/installs/ruby/3.0.4/lib/ruby/gem
s/3.0.0/gems/decidim-core-0.27.2/app/views/layouts/decidim/_js_configuration.html.erb (Duration: 7.
9ms | Allocations: 3939)
[2025-07-17 13:14:21 -0300] DEBUG --    Rendered /home/victor/.asdf/installs/ruby/3.0.4/lib/ruby/gem
s/3.0.0/gems/decidim-core-0.27.2/app/views/layouts/decidim/_decidim_javascript.html.erb (Duration:
21.2ms | Allocations: 13024)
[2025-07-17 13:14:21 -0300] DEBUG --    Rendered /home/victor/.asdf/installs/ruby/3.0.4/lib/ruby/gem
s/3.0.0/gems/decidim-core-0.27.2/app/views/layouts/decidim/_data_consent_warning.html.erb (Duration
: 0.4ms | Allocations: 131)
[2025-07-17 13:14:21 -0300] DEBUG --    Rendered /home/victor/.asdf/installs/ruby/3.0.4/lib/ruby/gem
s/3.0.0/gems/decidim-core-0.27.2/app/views/layouts/decidim/_application.html.erb (Duration: 304.7ms
| Allocations: 155555)
[2025-07-17 13:14:21 -0300] INFO --    Rendered layout layouts/decidim/participatory_process.html.er
b (Duration: 34288.4ms | Allocations: 23295245)
[2025-07-17 13:14:21 -0300] INFO --    Completed 200 OK in 34397ms (Views: 33328.2ms | ActiveRecord: 9
74.4ms | Allocations: 23345879)
```

Fonte: Próprio autor

5.5 Testes e Resultados

Os testes de carga serão executados para medir quantitativamente os resultados obtidos pelas modificações realizadas na ferramenta de textos participativos do Brasil Participativo. Para a execução dos testes, será utilizada a ferramenta de linha de comando *Apache HTTP Server Benchmarking Tool* (ApacheBench). Os testes serão executados em um computador com sistema operacional Linux, distribuição Ubuntu 24.04.2 LTS. As

configurações de *hardware* podem ser vistas na Tabela 3. Os testes utilizarão a aplicação do Brasil Participativo com execução em modo produção. O protocolo utilizado será HTTP, e o próprio *Rails* servirá os *assets*. O *web-server Puma* será executado em *single mode* com cinco *threads*.

Tabela 3 – Especificação do hardware de teste

Processador	AMD Ryzen 5 7430U (6 Cores/ 12 Threads)
Placa de Vídeo	Radeon Graphics 2GB (Integrado)
Memória RAM	24 GB DDR4

Os testes consistirão em submeter sob carga as páginas de visualização do texto participativo e visualização do rascunho (edição do texto participativo), dado que estas foram as páginas que receberam o foco para otimização de desempenho neste trabalho.

Serão levantados dados para comparação com a realidade do cotidiano e os resultados dos testes apresentados pelo ApacheBench, trazendo a noção do impacto causado pelas modificações deste realizadas. Para trazer variabilidade nos testes, serão executados tanto os testes com volumetria de parágrafos histórica, quanto com volumetria de parágrafos extremas.

Para ambos os cenários de teste, cada parágrafo terá seu conteúdo preenchido com um texto de exemplo "*Lorem Ipsum*" de 563 caracteres cada. O conteúdo utilizado será:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed in odio non mi auctor commodo. Aenean consequat metus massa, eget vulputate nibh suscipit vitae. Aliquam sagittis vel odio eleifend posuere. Curabitur hendrerit est elit, ac hendrerit nulla laoreet sit amet. Proin dictum tempor vulputate. Etiam cursus, quam sed cursus laoreet, dui nisi efficitur quam, at maximus dui nibh et velit. Curabitur ullamcorper ligula vitae mauris cursus rhoncus. Suspendisse laoreet felis euismod lorem accumsan, sit amet imperdiet mauris laoreet. Fusce vel tristique felis.

A ideia central é checar o quão bem a plataforma possivelmente operará no dia-dia e em cenários de alta demanda. Para os cenários extremos, será adotada a volumetria de 2 mil parágrafos por texto participativo, e 10 administradores utilizando a ferramenta paralelamente.

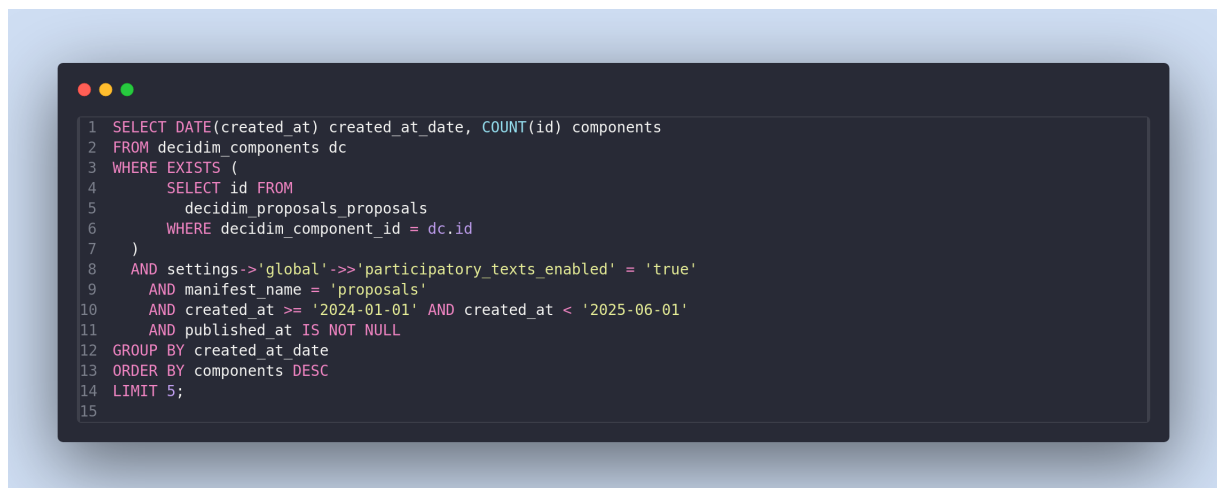
A versão anterior às modificações descritas neste trabalho utilizada para comparação nos testes é a *tag* 1.5.0, que está disponível no repositório oficial do Brasil Participativo.

5.5.1 Teste de Carga de Visualização do Rascunho

5.5.1.1 Contexto

A fim de obter um parâmetro para comparação com teste de carga da página de edição de rascunho, foi executada no ambiente produtivo do Brasil Participativo uma consulta SQL para descobrir o dia que houveram mais criações de componentes de texto participativo. A consulta considera o período entre o primeiro dia do ano de 2024 até o último dia do mês de maio de 2025, admitindo apenas componentes que foram publicados e que são textos participativos. A consulta SQL pode ser conferida na Figura 7, e o seu resultado é demonstrado na Tabela 4.

Figura 7 – Consulta SQL para classificação dos dias com maior carga de criação de textos participativos



Fonte: Próprio autor

Tabela 4 – Top 5 dias com maior volume de textos participativos criados

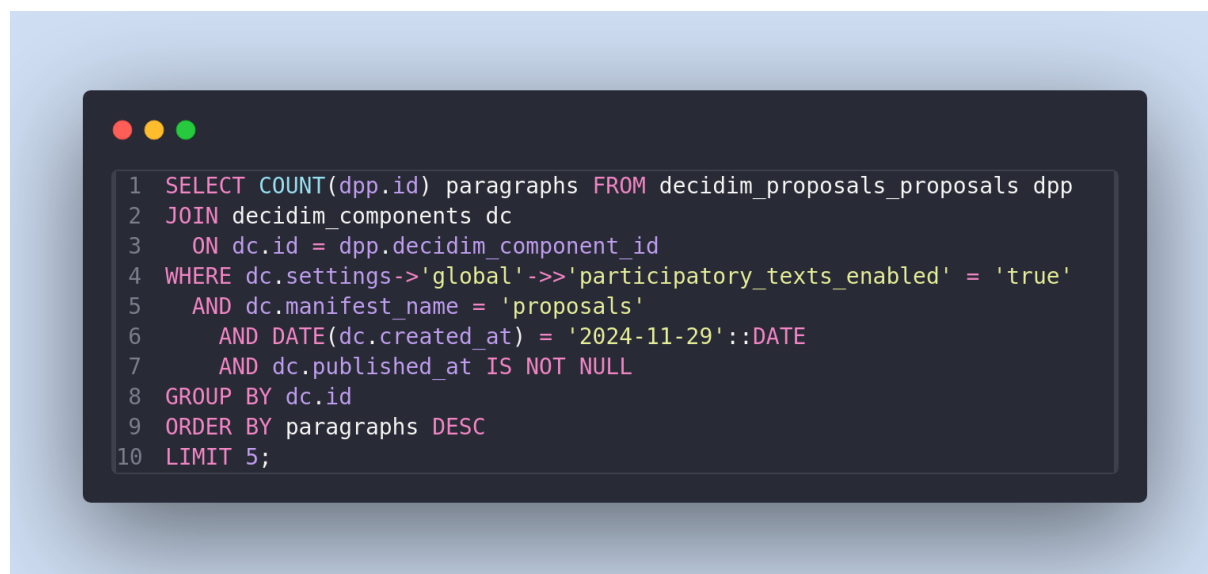
Data	Textos participativos criados
29/11/2024	45
30/11/2024	24
02/12/2024	17
22/01/2025	15
09/10/2024	12

Fonte: Próprio autor

No dia 29 de novembro de 2024, houveram quarenta e cinco (45) componentes de texto participativo criados na plataforma. Uma segunda consulta foi executada para descobrir a classificação dos cinco textos participativos dessa data com a maior quantidade de parágrafos. A consulta SQL pode ser conferida na Figura 8, e o seu resultado na Tabela

5. Nota-se que para essa data, o texto participativo com a maior quantidade de parágrafos foi de 124.

Figura 8 – Consulta SQL para classificação dos textos participativos com maior quantidade de parágrafos



Fonte: Próprio autor

Tabela 5 – Top 5 textos participativos de 29/11/2024 com maior volume de parágrafos

Parágrafos
124
85
80
76
74

Fonte: Próprio autor

Com base nessas informações, considerar-se-á um dia com pico de uso com 50 componentes sendo criados, cada um deles com 130 parágrafos. Considera-se nesse trabalho que o usuário administrador possivelmente trabalha na plataforma entre os horários das nove horas da manhã até meio dia (12 horas), e das 13 horas até às 18 horas, significando uma janela de trabalho de oito horas, o que significa em aproximadamente de seis a sete componentes criados por hora.

Assumindo um possível pior cenário, também por convenção, considerar-se-á neste trabalho que o administrador realiza em média no mínimo todas as cinco operações em cada parágrafo, isto é: editar, mesclar, mover, ocultar e desabilitar interações. Para este teste será também ignorado o fato de que a operação de mesclagem diminui a contagem de parágrafos em uma unidade. A realização de cada operação acarreta na atualização da

página de visualização do rascunho, portanto considerando os seis a sete componentes por hora (nesse caso será considerado sete, simplesmente), a quantidade de 150 parágrafos e as cinco operações por parágrafo, tem-se que a página de visualização do rascunho será renderizada 5.250 vezes em uma hora, ou 87 vezes por minuto, ou 1,46 vezes por segundo.

5.5.1.2 Execução e Resultados

Os testes foram realizados disparando 100 requisições com o fator de paralelismo igual a cinco. Os resultados podem ser visualizados na Tabela 6

Tabela 6 – Teste da página de visualização de rascunho

Métrica	Versão anterior	Nova versão
Tamanho da página	931.752 bytes	712.295 bytes
Tempo gasto no teste	109 segundos	18 segundos
Requisições por segundo	0,91	5,38
Tempo médio por requisição	1.097 milissegundos	185 milissegundos

Fonte: Próprio autor

5.5.1.3 Testes Extremos

Os testes extremos foram realizados com 10 requisições com fator de paralelismo igual a 10. O número de requisições foi escolhido como 10 para garantir que os testes ocorressem em uma escala de tempo na casa de alguns minutos, evitando problemas de expiração de sessão, dado que o teste foi realizado usando um Cookie de sessão de usuário administrador extraído do navegador manualmente. O texto participativo utilizado possui 2 mil parágrafos. Os resultados podem ser conferidos na Tabela 7.

Tabela 7 – Teste extremo da página de visualização de rascunho

Métrica	Versão anterior	Nova versão
Tamanho da página	14.027.611 bytes	5.888.914 bytes
Tempo gasto no teste	158 segundos	20 segundos
Requisições por segundo	0,06	0,48
Tempo médio por requisição	15.837 milissegundos	2.084 milissegundos

Fonte: Próprio autor

5.5.2 Teste de Carga de Visualização do Texto

5.5.2.1 Contexto

No cenário de visualização do texto participativo, não há engajamento notório de usuários na plataforma. As contribuições se limitam a algumas dezenas de comentários em

todo o ciclo de vida do texto. Pode-se visualizar na Figura ?? uma consulta SQL utilizada para verificar o dia em que mais foram submetidas contribuições a textos participativos. Dessa forma, optou-se por manter o padrão de carga de teste utilizado na página de visualização do rascunho.

5.5.2.2 Execução e Resultados

Os testes foram realizados disparando 100 requisições com o fator de paralelismo igual a cinco. Os resultados podem ser visualizados na Tabela 8. Para esse teste não se faz necessário o uso de Cookie de sessão de usuário, uma vez que a página é pública e possível de visualizar sem estar logado na plataforma.

Tabela 8 – Teste da página de visualização do texto

Métrica	Versão anterior	Nova versão
Tamanho da página	437.372	528.236
Tempo gasto no teste	455 segundos	15 segundos
Requisições por segundo	0,22	6,61
Tempo médio por requisição	4.551 milissegundos	151 milissegundos

Fonte: Próprio autor

5.5.2.3 Testes Extremos

Tabela 9 – Teste extremo da página de visualização do texto

Métrica	Versão anterior	Nova versão
Tamanho da página	2.510.864 bytes	3.239.958 bytes
Tempo gasto no teste	503 segundos	16 segundos
Requisições por segundo	0,02	0,61
Tempo médio por requisição	50.345 milissegundos	1.644 milissegundos

Fonte: Próprio autor

Referências

- ADAM, B. M.; BESARI, A. R. A.; BACHTIAR, M. M. Backend server system design based on rest api for cashless payment system on retail community. In: IEEE. *2019 International Electronics Symposium (IES)*. [S.l.], 2019. p. 208–213. Citado na página 27.
- CARLSON, J. L. *Redis in Action*. Shelter Island, NY: Manning Publications Co., 2013. ISBN 9781935182054. Citado na página 29.
- CONALLEN, J. Modeling web application architectures with uml. *Communications of the ACM*, ACM New York, NY, USA, v. 42, n. 10, p. 63–70, 1999. Citado na página 26.
- FRY, J. P.; SIBLEY, E. H. Evolution of data-base management systems. *ACM Computing Surveys (CSUR)*, ACM New York, NY, USA, v. 8, n. 1, p. 7–42, 1976. Citado na página 28.
- GONG, Y. et al. The architecture of micro-services and the separation of frond-end and back-end applied in a campus information system. In: IEEE. *2020 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA)*. [S.l.], 2020. p. 321–324. Citado na página 27.
- GOUGH, J.; BRYANT, D.; AUBURN, M. *Mastering API Architecture*. [S.l.]: "O'Reilly Media, Inc.", 2021. Citado 2 vezes nas páginas 23 e 27.
- GROUP, P. G. D. *About PostgreSQL*. 2023. Accessed on December 4, 2023. Disponível em: <<https://www.postgresql.org/about/>>. Citado na página 29.
- JATANA, N. et al. A survey and comparison of relational and non-relational database. *International Journal of Engineering Research & Technology*, v. 1, n. 6, p. 1–5, 2012. Citado na página 28.
- JUGO, I.; KERMEK, D.; MEŠTROVIĆ, A. Analysis and evaluation of web application performance enhancement techniques. In: SPRINGER. *Web Engineering: 14th International Conference, ICWE 2014, Toulouse, France, July 1-4, 2014. Proceedings 14*. [S.l.], 2014. p. 40–56. Citado na página 33.
- METADECIDIM. *Decidim Documentation*. 2023. Acesso em: 10 de dezembro de 2023. Disponível em: <<https://docs.decidim.org/en/v0.27/index.html>>. Citado na página 37.
- METADECIDIM. *General description and introduction to how Decidim works*. 2023. Acesso em: 10 de dezembro de 2023. Disponível em: <<https://docs.decidim.org/en/v0.27/features/general-description>>. Citado 2 vezes nas páginas 38 e 39.
- MODEL-VIEW-CONTROLLER Pattern. In: LEARN Objective-C for Java Developers. Berkeley, CA: Apress, 2009. p. 353–402. ISBN 978-1-4302-2370-2. Disponível em: <https://doi.org/10.1007/978-1-4302-2370-2_20>. Citado 3 vezes nas páginas 23, 29 e 30.
- OLUWATOSIN, H. S. Client-server model. *IOSR Journal of Computer Engineering*, IOSR Journals, v. 16, n. 1, p. 67–71, 2014. Citado na página 27.

- PARTICIPATIVO, B. *Sobre o Brasil Participativo*. 2023. Acesso em: 10 de dezembro de 2023. Disponível em: <<https://brasilparticipativo.presidencia.gov.br/processes/brasilparticipativo/f/33>>. Citado na página 41.
- TANENBAUM, A. S.; KLINT, P.; BOHM, W. Guidelines for software portability. *Software: Practice and Experience*, Wiley Online Library, v. 8, n. 6, p. 681–698, 1978. Citado 2 vezes nas páginas 23 e 26.
- THE RAILS FOUNDATION. *Action Controller Overview*. 2023. Acesso em: 9 de dezembro de 2023. Disponível em: <https://guides.rubyonrails.org/action_controller_overview.html>. Citado na página 32.
- THE RAILS FOUNDATION. *Action View Overview*. 2023. Acesso em: 9 de dezembro de 2023. Disponível em: <https://guides.rubyonrails.org/action_view_overview.html>. Citado na página 32.
- THE RAILS FOUNDATION. *Actionpack Action Caching*. 2023. Acesso em: 12 de dezembro de 2023. Disponível em: <https://github.com/rails/actionpack-action_caching>. Citado na página 34.
- THE RAILS FOUNDATION. *Actionpack Page Caching*. 2023. Acesso em: 12 de dezembro de 2023. Disponível em: <https://github.com/rails/actionpack-page_caching>. Citado na página 34.
- THE RAILS FOUNDATION. *Active Model Basics*. 2023. Acesso em: 9 de dezembro de 2023. Disponível em: <https://guides.rubyonrails.org/active_model_basics.html>. Citado na página 31.
- THE RAILS FOUNDATION. *Active Record Basics*. 2023. Acesso em: 9 de dezembro de 2023. Disponível em: <https://guides.rubyonrails.org/active_record_basics.html>. Citado na página 31.
- THE RAILS FOUNDATION. *Caching with Rails: An Overview*. 2023. Acesso em: 12 de dezembro de 2023. Disponível em: <https://guides.rubyonrails.org/caching_with_rails.html>. Citado 3 vezes nas páginas 33, 34 e 35.
- THE RAILS FOUNDATION. *Ruby on Rails API*. 2023. Acesso em: 9 de dezembro de 2023. Disponível em: <<https://api.rubyonrails.org/>>. Citado 2 vezes nas páginas 23 e 31.
- THE RAILS FOUNDATION. *Debugging Rails Applications*. 2025. Acesso em: 17 de julho de 2025. Disponível em: <https://guides.rubyonrails.org/debugging_rails_applications.html>. Citado na página 53.
- WILLIAMS, L. G.; SMITH, C. U. Web application scalability: A model-based approach. In: *Int. CMG Conference*. [S.l.: s.n.], 2004. p. 215–226. Citado na página 33.
- YU, S. Acid properties in distributed databases. *Advanced eBusiness Transactions for B2B-Collaborations*, p. 17, 2009. Citado na página 28.